

Documentation technique — Vite & Gourmand

Architecture, sécurité, données, déploiement et vérifications

1. Présentation du projet

Vite & Gourmand est une application web de restauration et de service traiteur permettant aux visiteurs de consulter les menus proposés par l'entreprise et aux utilisateurs authentifiés de passer et suivre leurs commandes.

L'application distingue trois rôles principaux : utilisateur, employé et administrateur. Le projet utilise MySQL pour les données métier et MongoDB pour les statistiques administratives.

2. Choix technologiques

2.1 Front-end

HTML5, CSS3 et JavaScript natif. Le site possède notamment une navigation responsive avec menu burger, des pages de consultation et de détail des menus, des filtres dynamiques, des formulaires et des espaces dédiés aux utilisateurs, employés et administrateurs.

2.2 Back-end

Le back-end est développé en PHP. PDO est utilisé pour communiquer avec MySQL. Composer gère notamment l'API Brevo via HTTPS/cURL et vlucas/phpdotenv.

2.3 Base de données relationnelle

MySQL stocke les utilisateurs et leurs rôles, menus, plats, images, associations menus/plats, commandes, historiques, annulations, avis, horaires, demandes de contact et jetons de réinitialisation.

2.4 Base de données NoSQL

MongoDB est utilisé dans l'espace administrateur pour stocker et exploiter les statistiques liées aux menus et aux commandes. L'application communique avec MongoDB grâce à l'extension PHP mongodb.

2.5 Envoi des e-mails

Les e-mails applicatifs sont envoyés via l'API Brevo appelée en HTTPS avec cURL depuis le back-end PHP. Ils concernent notamment les confirmations de commande, réinitialisations de mot de passe, demandes de contact, créations de comptes employés, annulations et restitution de matériel.

2.6 Calcul des frais de livraison

Le calcul utilise OpenRouteService. La livraison possède un tarif de base de 5 €. Pour une livraison en dehors de Bordeaux, un supplément de 0,59 € par kilomètre est calculé à partir de la distance

routière.

2.7 Déploiement

L'application est déployée avec Railway. MySQL de production est hébergé sur Railway et la base NoSQL sur MongoDB Atlas. Le code source est versionné avec Git et hébergé sur GitHub.

3. Environnement de travail

Le développement a été réalisé dans un environnement local avant la mise en production.

3.1 Outils de développement

- Visual Studio Code
- PHP
- Composer
- MySQL
- MongoDB
- Git
- GitHub
- Figma
- GitHub Projects

3.2 Serveur de développement

L'application peut être lancée depuis la racine du projet avec `php -S localhost:8000` et est alors accessible sur `http://localhost:8000`.

3.3 Gestion des dépendances

Les dépendances sont définies dans `composer.json` et `composer.lock`. Elles peuvent être installées avec `composer install`. L'extension PHP mongodb est nécessaire à la communication avec MongoDB.

3.4 Configuration de l'environnement

Le fichier `.env` contient les valeurs propres à l'environnement d'exécution tandis que `.env.example` documente les variables nécessaires sans contenir les secrets. Les principales variables sont `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `SMTP_FROM_EMAIL`, `BREVO_API_KEY`, `MONGODB_URI`, `APP_URL` et `ORS_API_KEY`. Le fichier `.env` est exclu du dépôt Git.

4. Modélisation

4.1 Modèle conceptuel de données

La base relationnelle est organisée autour des utilisateurs, menus, plats, images des menus, commandes, historiques de statuts, annulations, avis, horaires, demandes de contact et jetons de réinitialisation. La relation entre menus et plats est plusieurs-à-plusieurs via `menu_dishes`.

```
USERS ||--o{ ORDERS
USERS ||--o{ REVIEWS
MENUS ||--o{ MENU_IMAGES
MENUS ||--o{ MENU_DISHES
DISHES ||--o{ MENU_DISHES
MENUS ||--o{ ORDERS
ORDERS ||--o{ ORDER_STATUS_HISTORY
ORDERS ||--o| ORDER_CANCELLATIONS
```

4.2 Diagramme de cas d'utilisation

Les droits d'accès sont organisés selon les rôles visiteur, utilisateur, employé et administrateur. Les principales fonctions couvrent la consultation et le filtrage des menus, l'inscription, la connexion, le contact, la commande, le suivi, les avis, la gestion des menus et commandes, la modération, les horaires, les employés et les statistiques.

4.3 Diagramme de séquence — Passage d'une commande

Le parcours principal vérifie l'utilisateur, le menu et le stock, contrôle le nombre minimum et le délai de préparation, applique la remise si éligible, calcule la livraison, enregistre la commande et le stock, puis déclenche l'e-mail de confirmation.

```
Utilisateur -> Front-end -> Back-end PHP -> MySQL
Back-end -> Service de livraison
Back-end -> Service e-mail
Back-end -> Front-end : commande créée
```

5. Sécurité

5.1 Mots de passe

Les mots de passe respectent les règles de l'application, ne sont pas stockés en clair et sont transformés avec `password_hash()`. La réinitialisation utilise un jeton temporaire.

5.2 Protection contre les injections SQL

Les accès MySQL utilisent PDO et des requêtes préparées.

5.3 Validation des données

Le back-end contrôle notamment le nombre minimum de personnes, la remise de 10 %, les dates et horaires de livraison, les délais de préparation, la disponibilité et les droits de modification ou d'annulation.

5.4 Gestion des rôles et des accès

L'application distingue user, employee et admin. Les routes sensibles contrôlent la session et le rôle.

5.5 Protection des informations sensibles

Les identifiants de bases de données, clés Brevo et OpenRouteService sont stockés dans des variables d'environnement. `.env` est exclu du dépôt Git.

5.6 Données personnelles

L'application ne collecte que les informations nécessaires au fonctionnement du service. Les données personnelles ne doivent pas être exposées publiquement.

5.7 Accessibilité

Une attention est portée à la structure HTML sémantique, aux textes alternatifs, aux attributs ARIA lorsque nécessaires, à la navigation responsive et à la lisibilité. Ces dispositions participent à la prise en compte du RGAA sans constituer une certification complète.

6. Déploiement

L'application est déployée sur Railway à partir du dépôt GitHub.

6.1 Préparation

- Dépendances déclarées dans `composer.json` et verrouillées dans `composer.lock`.
- Extension PHP mongodb déclarée comme dépendance de plateforme.
- Secrets retirés du code source et placés dans des variables d'environnement.
- `.env` exclu du dépôt Git.
- `.env.example` utilisé pour documenter les variables nécessaires.

6.2 Base MySQL

La structure et les données initiales sont installées avec `database/sql/01_creation_tables.sql` puis `database/sql/02_insert_data.sql`. Les scripts ont été testés sur une base MySQL vierge.

6.3 Base MongoDB

La base NoSQL de production est hébergée sur MongoDB Atlas et la connexion utilise `MONGODB_URI`.

6.4 Variables d'environnement

Les variables principales de production sont `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `SMTP_FROM_EMAIL`, `BREVO_API_KEY`, `MONGODB_URI`, `APP_URL` et `ORS_API_KEY`. Elles sont configurées dans Railway et ne sont pas enregistrées dans Git.

6.5 Services externes

- Brevo pour les e-mails transactionnels via son API HTTPS.
- OpenRouteService pour le calcul des distances de livraison.
- MongoDB Atlas pour la base NoSQL.

6.6 Mise en ligne

Railway est relié au dépôt GitHub du projet et la branche de production est `main`. Une adresse HTTPS publique est générée par Railway. `APP_URL` contient l'adresse publique de l'application.

6.7 Vérifications après déploiement

Des tests fonctionnels ont été réalisés sur l'environnement de production : pages publiques et six menus, inscription, connexion, réinitialisation, menus, commande et minimum de personnes, remise de 10 %, frais de livraison, suivi jusqu'à `completed`, e-mails Brevo, restitution du matériel, avis, espaces employé et administrateur, MySQL, MongoDB et statistiques.

7. Conclusion technique

Le développement a permis de mettre en place une application web complète intégrant une interface responsive, une gestion des utilisateurs et des rôles, un système de commande, une gestion des menus et des plats, ainsi que des espaces employés et administrateurs.

L'architecture repose sur PHP pour le back-end, HTML/CSS/JavaScript pour le front-end, MySQL pour les données métier et MongoDB pour les statistiques. Les informations sensibles sont séparées dans des variables d'environnement.